UK2ME ONLINE

Milestone 2 — Work Plan

Location-based baskets, admin invoicing & end-to-end email notifications

Project: UK2ME Revamp (monorepo — apps/client · apps/admin · apps/backend) · Prepared: 29 May 2026 · Revised: 2 June 2026 · Status: Planning → ready to build

Revision (2 Jun 2026): Adds three deliverables from the latest client notes — a configurable **Delivery Engine** (Thursday despatch + multi-leg date estimates), an **Order Dispute & Refund** flow with email notifications, and a **Delivery Note shown before payment**. These are **R6–R8** (§§4.5–4.7, WS-7–WS-9).

1. Purpose

The live UK2ME flow is a manual personal-shopping service: a customer pastes UK/US product links, places an order, and staff then build a priced invoice **by hand in Excel** — grouping items by store, adding store postage, transfer fees, service charge, and weight-based Nigeria postage. Two problems were confirmed on the walkthrough call:

1. **Invoicing is fully manual** and lives outside the system.
2. **Email notifications are unreliable / not arriving**, so staff fall back to Excel and phone calls.

This milestone closes those gaps with five concrete deliverables.

2. Requirements (this milestone)

#	REQUIREMENT	SOURCE
R1	**Location-based baskets** — separate UK (£) and US ($) carts. A customer cannot mix regions in one basket because pricing rules differ.	Call + message
R2	**US has a minimum service charge; UK does not.** Service-charge rule is region-specific (e.g. US flat charge when order value is below a threshold).	Message
R3	**Order-placed notification to admin, including the product link**, so staff can review the order themselves before pricing.	Message
R4	**Admin creates an invoice from the order and sends it to the client.** The order stays **pending** until the client pays.	Message
R5	**Every state change sends an email** — to customer and, where relevant, admin. Fix the underlying non-delivery problem.	Call + message
R6	**Delivery Engine** — estimated despatch + delivery dates from configurable rules: in-country leg (store→hub), a **weekly Thursday despatch** from UK & US hubs, and the international leg (hub→Lagos). **Two independent speeds** (Leg-1 & Leg-2), **Express UK-only**.	2 Jun notes
R7	**Order Dispute & Refund** — client button to raise a dispute / request a refund; admin reviews and resolves; **email notifications** at every step. Refund money moved **manually** in the gateway; the app records the state.	2 Jun notes
R8	**Delivery Note before payment** — the engine's estimate (legs, speeds, despatch day, delivery window) shown to the customer **before they pay**, **persisted on the order** and **visible in admin**.	2 Jun notes

3. Current state vs. gap

AREA	ALREADY BUILT	GAP TO CLOSE
Checkout	Single **GBP-only** checkout (`/api/v1/checkout`)	Region-aware UK/US baskets & orders (R1)
Pricing	Weight engine, FX overrides, tax %, delivery/logistic fees in `AppSetting`	Region service-charge rule + per-store-group invoice math (R2)
Orders	`Order` + statuses PLACED…DELIVERED; admin list exposes `productUrl`	New statuses for invoice/payment lifecycle (R4)
Notifications	Customer emails (order received / status / shipped); mailer **swallows errors**	Admin order-placed email; invoice email; reliable delivery + logging (R3, R5)
Invoicing	Client Invoices page is a **placeholder**; no `Invoice` model; no admin builder	Full invoice model, admin builder UI, client view, PDF/email (R4)
Delivery timing	Checkout has only `door` / `pickup` flat fees — **no date estimation at all**	Configurable multi-leg engine: Thursday despatch + delivery window (R6, R8)
Disputes/refunds	**No dispute or refund concept** in model or UI	`Dispute` model, client raise-button, admin resolve, refund states + emails (R7)

4. Data model changes (prisma/schema.prisma)

4.1 Order status enum — extend lifecycle so an order can sit "pending" after invoicing:

```
PLACED              // customer submitted, awaiting admin review
PENDING_INVOICE     // admin reviewing / building invoice
INVOICED            // invoice sent, awaiting client payment  ← "pending"
PROCESSING          // payment confirmed, sourcing item
AWAITING_PURCHASE → SHIPPED → DELIVERED → CANCELLED   (unchanged)
```

4.2 Add `region` to `Order` — `UK` | `US` (drives currency, service-charge rule, basket).

4.3 New `Invoice` + `InvoiceLineItem` models:

```
Invoice
  id, orderId (1:1), invoiceNumber, region (UK|US), currency
  status: DRAFT | SENT | PAID | VOID
  itemsSubtotal, storePostage, salesTax,
  internationalTransferFee, serviceCharge,
  nigeriaPostage, domesticPostage, total
  fxRateSnapshot, notes
  sentAt, dueAt, paidAt, createdAt, updatedAt
  lineItems  InvoiceLineItem[]

InvoiceLineItem
  id, invoiceId, storeName (group key), productTitle, productUrl,
  size, color, qty, unitPrice, lineTotal, weightGrams
```

Line items are **grouped by store** (all Amazon together, all Aldo together) to mirror how staff already build invoices.

4.4 Service-charge settings (in `AppSetting`, region-keyed): `service_charge_us_min`, `service_charge_us_threshold`, `service_charge_uk_*` (UK = 0 by default).

4.5 Delivery Engine — config + persisted estimate (R6, R8). All timings live in `AppSetting` (admin-editable, no redeploy), with `lib/settings.ts` getters:

```
delivery_processing_days        // order placed → ready for Leg 1   (default 1, working days)
delivery_leg1_std_min/max       // store → hub, standard            (default 3 / 5 days)
delivery_leg1_express_min/max   // store → hub, express (UK only)   (default 1 / 2 days)
delivery_despatch_weekday       // weekly despatch day              (default 4 = Thursday)
delivery_despatch_cutoff_days   // must be at hub N days before      (default 1 → by Wednesday)
delivery_leg2_std_min/max       // hub → Lagos, standard            (default 5 / 10 WORKING days)
delivery_leg2_express_min/max   // hub → Lagos, express (UK only)   (default 2 / 3 working days)
delivery_express_regions        // regions allowing express         (default "UK")
```

Persisted on `Order` (snapshot at quote-time so estimates don't drift):

```
Order  + leg1Speed (STD|EXPRESS)  + leg2Speed (STD|EXPRESS)
       + despatchDate  + estDeliveryMin  + estDeliveryMax  + deliveryQuotedAt
       + deliveryNote  Json   // full leg-by-leg breakdown for the Delivery Note & admin
```

Engine (`lib/delivery-engine.ts` , pure + unit-testable; types mirrored into `packages/shared`):

```
estimateDelivery({ placedAt, region, leg1Speed, leg2Speed }) → {
  readyAt      = addWorkingDays(placedAt, processingDays)
  hubArriveMin = addDays(readyAt, leg1Min);  hubArriveMax = addDays(readyAt, leg1Max)
  despatchDate = nextDespatchDay(hubArriveMax + cutoff, weekday)   // roll to the Thursday
  deliveryMin  = addWorkingDays(despatchDate, leg2Min)
  deliveryMax  = addWorkingDays(despatchDate, leg2Max)
  → { despatchDate, deliveryMin, deliveryMax, legs[…], speedLabels, expressAvailable }
}
```

Express is honoured only when `region ∈ delivery_express_regions` (UK); otherwise it falls back to standard with a notice. A shared `addWorkingDays()` helper skips weekends (NG/UK public-holiday calendars can be layered on later).

4.6 Dispute & Refund (R7) — new enum + model, and lifecycle additions:

```
enum DisputeStatus { OPEN  UNDER_REVIEW  RESOLVED  REJECTED }
OrderStatus   += DISPUTED, REFUNDED
PaymentStatus += REFUNDED

model Dispute
  id, orderId, userId, reason, detail
  requestedRefund Boolean, refundAmount Float?
  status DisputeStatus @default(OPEN)
  resolutionNote, refundedAt, refundedBy
  createdAt, updatedAt, resolvedAt
```

Refund execution is **manual** (admin moves money in the Paystack/Stripe dashboard); the app records `Payment.status=REFUNDED` , `Order.status=REFUNDED` , and an `OrderEvent` for audit.

4.7 Delivery Note (R8) — *not a new model*; it is the rendered view of the §4.5 estimate stored on the `Order` (`deliveryNote` JSON + the est. fields), shown as a block on the pre-payment / invoice screen and on the admin order detail.

5. Workstreams

*Each task lists a **verify** check so progress is testable, not "looks done".*

WS-1 — Region-aware baskets R1

~2 days

1. Add `region` to client cart store (Zustand) → one active region per basket; switching region warns/empties.
   verify: cannot add a US link to a UK basket; UI reflects £ vs $.
2. `region` carried through `/api/v1/checkout`; order persisted with region + correct currency (remove hardcoded `'GBP'`).
   verify: a US order is stored with region=US, currency=USD.
3. Client checkout + dashboard show the right currency/flag per order.
   verify: order list shows £ and $ correctly.

WS-2 — Invoice data model & engine R2 R4

~3 days

1. Migrations for §4 (Order status, region, Invoice models, settings).
   verify: prisma migrate clean; prisma generate types compile.
2. Invoice-calc service: group items by store; apply store postage, **US sales tax**, international transfer fee, **region service charge (US min / UK none)**, weight-based Nigeria postage (reuse `lib/weight.ts`), optional Lagos→outside domestic postage.
   verify: unit tests for US-below-threshold (charge applied) vs UK (none), plus a multi-store order totalling correctly.
3. Admin endpoints: `POST …/orders/[id]/invoice` (draft), `PATCH …/invoice` (edit), `POST …/invoice/send`.
   verify: creating an invoice flips order to INVOICED; refetch returns saved breakdown.

WS-3 — Admin invoice builder UI R4

~2.5 days

1. On order detail page, "Build invoice" panel: auto-grouped store sections, editable line prices, fee fields pre-filled from settings, live total.
   verify: editing a fee updates total instantly; matches engine output.
2. "Send invoice" → sets invoice SENT, order INVOICED (pending), triggers client email.
   verify: client receives invoice email; order shows pending.

WS-4 — Client invoice view R4

~1.5 days

1. Replace placeholder Invoices page with real list + detail (store-grouped breakdown, total, status).
   verify: client sees the sent invoice with full breakdown.
2. "Pay now" wires to existing Paystack/Stripe flow using invoice total; on payment, order → PROCESSING, invoice → PAID.
   verify: simulated payment moves both states and emails fire.

WS-5 — Notifications: admin alerts + reliability R3 R5

~2 days

1. **Admin order-placed email** on checkout — customer, items, and **product link(s)** for self-review.
   verify: placing an order emails the admin address with clickable product links.
2. New templates: admin order alert, invoice ready (client), invoice paid (client + admin); extend status emails to new statuses.
   verify: each state change produces the matching email.
3. **Mailer reliability:** stop silently swallowing failures — log + retry once, record send status on `OrderEvent`, surface failures in admin. Document SMTP config (Gmail app-password on file).
   verify: a forced SMTP error is logged and visible; a healthy send is recorded.
4. Admin recipient configurable via `AppSetting` (`admin_notify_email`).
   verify: changing the setting reroutes alerts.

WS-6 — Invoice PDF + polish

~1 day (optional)

1. Render invoice to PDF (HTML→PDF), attach to invoice email, downloadable by client.
   verify: emailed/downloaded PDF matches on-screen breakdown.

WS-7 — Delivery Engine + Delivery Note R6 R8

~3 days

1. `lib/delivery-engine.ts` pure function + `addWorkingDays` / `nextDespatchDay` helpers; seed §4.5 settings with defaults.
   *verify: unit tests — an order placed Mon (std/std, UK) despatches the **same-week Thursday**; one placed so Leg-1 misses cutoff rolls to **next Thursday**; Leg-2 window counts **working** days only.*
2. Settings getters + admin "Delivery timings" panel (processing, leg min/max, despatch weekday, cutoff, express regions).
   verify: changing `delivery_leg2_std_max` shifts quoted windows without redeploy.
3. Migrate `Order` delivery fields; compute + snapshot the estimate at quote/checkout.
   verify: order row stores `despatchDate`, `estDeliveryMin/Max`, `deliveryNote` JSON.
4. **Delivery Note** block on the pre-payment / invoice screen (region, two speed pickers with **Express gated to UK**, despatch Thursday, delivery window) and on admin order detail.
   verify: a US order hides express; the on-screen note matches the stored snapshot and admin view.

WS-8 — Order Dispute & Refund R7

~2.5 days

1. Migrate §4.6 (`Dispute` model, `DisputeStatus`, `OrderStatus += DISPUTED/REFUNDED`, `PaymentStatus += REFUNDED`).
   verify: prisma migrate clean; types compile.
2. Client **"Raise a dispute / Request refund"** on order detail (eligible statuses only) → form (reason + detail + optional refund) → creates `Dispute(OPEN)`, emails customer confirmation + admin alert.
   verify: submitting creates the dispute and both emails fire.
3. Admin **Disputes** list + detail → set `UNDER_REVIEW`, then **Resolve** (optionally approve refund → `Payment=REFUNDED`, `Order=REFUNDED`, record `refundedAt/By`) or **Reject**; each transition emails the customer.
   verify: approving a refund flips both states, writes an `OrderEvent`, and emails the customer.

WS-9 — Delivery/dispute notifications wiring R5 R7

folded in

1. New templates: **dispute opened** (customer + admin), **dispute status changed / resolved / rejected** (customer), **refund processed** (customer + admin) — sent via the WS-5 reliability path.
   verify: each transition produces the matching email, logged.

6. Email notification matrix R5

EVENT	CUSTOMER	ADMIN
Account verify / password reset	✓	—
Order placed	✓ (received)	✓ **(with product link)**
Invoice sent	✓ (invoice + breakdown)	—
Payment received	✓	✓
Status: sourcing / shipped / delivered / cancelled	✓	—
Dispute opened	✓ (received)	✓ (alert)
Dispute status changed / resolved / rejected	✓	—
Refund processed	✓	✓

7. Sequencing & estimate

```
WS-1 baskets ─┐
WS-2 model/engine ─┼─► WS-3 admin builder ─► WS-4 client view ─► WS-6 PDF
WS-5 notifications ─┘  (parallel; touches WS-3/WS-4 hooks)

WS-7 delivery engine ──► Delivery Note on WS-4 pre-pay screen   (independent; can start now)
WS-8 dispute & refund ─► WS-9 dispute/refund emails (via WS-5 path)
```

Indicative effort: ~11–12 working days for R1–R5 (WS-6 optional) **+ ~5–6 days for R6–R8** (WS-7 ~3, WS-8 ~2.5, WS-9 folded in) → **~17–18 working days total**. WS-7 (delivery engine) is pure logic and can start immediately in parallel; WS-8 shares the WS-2 migration window.

8. Acceptance criteria (definition of done)

- ☐ A customer keeps **separate UK and US baskets**; orders persist the correct region & currency.
- ☐ Placing an order emails the **admin with the product link(s)**.
- ☐ Admin can **build an invoice** (store-grouped, region-correct fees, **US min service charge, no UK service charge**) and **send it**; the order shows **pending** until paid.
- ☐ The client sees the invoice with a full breakdown and can pay; payment moves the order to processing.
- ☐ **Every** lifecycle event sends the correct email; send successes/failures are **logged and visible** (no silent drops).
- ☐ The **Delivery Engine** quotes a despatch Thursday + delivery window from order date, region, and two speeds; **Express shows only for UK**; admin can edit timings in settings.
- ☐ The **Delivery Note** appears before payment, is stored on the order, and matches the admin view.
- ☐ A client can **raise a dispute / request a refund**; admin can **resolve or reject**; a marked refund sets `Order/Payment = REFUNDED`; every step emails the right parties.

9. Open questions (confirm before/early in build)

1. **Service-charge numbers** — confirm the US minimum amount and the order-value threshold (call mentioned "$15 when under $150"). UK confirmed = none.
2. **International transfer fee & Nigeria postage rate** — confirm current figures (in Excel today; we will seed them into settings).
3. **Domestic (Lagos → outside Lagos) postage** — flat fee, per-kg, or per-zone?
4. **Admin notification address(es)** — which inbox(es) receive order alerts?
5. **Does an invoice ever cover multiple orders**, or strictly one invoice per order? (Plan assumes 1:1.)
6. **Processing time & Thursday cutoff** — confirm the default `delivery_processing_days` (assumed **1**) and the cutoff for making that week's Thursday despatch (assumed **at the hub by Wednesday**). Are Leg-1 (store→hub) days **calendar** or **working** days? (Leg-2 is confirmed *working* days.)
7. **Express scope** — confirmed **UK-only** and **two independent legs**. Confirm US has **no** express on either leg, and whether express is **customer-selectable at checkout** or **admin-set / on request**.
8. **Dispute eligibility & refunds** — which order statuses can a customer dispute (e.g. after payment only, up to N days post-delivery)? Are **partial** refunds allowed, or full-only?
